Kalman Associative Memory Derivations

1 Kalman Associative Memory: optimal update

Let the associative-memory state be

$$\boldsymbol{S}_t \in \mathbb{R}^{d_k \times d_v}, \qquad \mathcal{M}_t(\boldsymbol{k}) = \boldsymbol{S}_t^\intercal \boldsymbol{k}.$$

The linear–Gaussian state-space model is

$$\boldsymbol{S}_t^\star = \boldsymbol{D}_t \boldsymbol{S}_{t-1}^\star + \boldsymbol{W}_t, \qquad \boldsymbol{W}_t \sim \mathcal{N}(0, \boldsymbol{\Omega}_t), \tag{1}$$

$$\boldsymbol{v}_t = \boldsymbol{S}_t^{\star\intercal} \boldsymbol{k}_t + \boldsymbol{e}_t, \qquad \boldsymbol{e}_t \sim \mathcal{N}(0, r_t \boldsymbol{I}). \tag{2}$$

Let $\boldsymbol{z}_s = (\boldsymbol{k}_s, \boldsymbol{v}_s)$ denote the key–value observation at step s. At the end of step $t-1$, the Kalman filter represents the posterior over the latent memory as a Gaussian,

$$p(\boldsymbol{S}_{t-1}^\star \mid \boldsymbol{z}_{1:t-1}) = \mathcal{N}(\boldsymbol{S}_{t-1}, \boldsymbol{P}_{t-1}), \tag{3}$$

where $\boldsymbol{S}_{t-1}$ is the posterior mean, i.e. the point estimate of the memory after processing the first $t-1$ key–value pairs, and $\boldsymbol{P}_{t-1}$ is its key-space uncertainty. For the matrix memory, this covariance is understood per value coordinate, with the same key-space covariance shared across value columns.

Before observing token t, the filter first predicts the next latent memory by marginalizing over the previous posterior:

$$p(\boldsymbol{S}_t^\star \mid \boldsymbol{z}_{1:t-1}) = \int p(\boldsymbol{S}_t^\star \mid \boldsymbol{S}_{t-1}^\star) p(\boldsymbol{S}_{t-1}^\star \mid \boldsymbol{z}_{1:t-1})\, d\boldsymbol{S}_{t-1}^\star. \tag{4}$$

Because both factors are linear–Gaussian, the predicted distribution is Gaussian,

$$p(\boldsymbol{S}_t^\star \mid \boldsymbol{z}_{1:t-1}) = \mathcal{N}(\widehat{\boldsymbol{S}}_t, \widehat{\boldsymbol{P}}_t). \tag{5}$$

Its mean and covariance are the exact Kalman prediction:

$$\widehat{\boldsymbol{S}}_t = \boldsymbol{D}_t \boldsymbol{S}_{t-1}, \qquad \widehat{\boldsymbol{P}}_t = \boldsymbol{D}_t \boldsymbol{P}_{t-1} \boldsymbol{D}_t^\intercal + \boldsymbol{\Omega}_t. \tag{6}$$

Thus $\widehat{\boldsymbol{S}}_t$ is a point estimate, but specifically the *predicted prior mean* of $\boldsymbol{S}_t^\star$ conditioned on the past key–value sequence $\boldsymbol{z}_{1:t-1}$. It is not yet the point estimate of $p(\boldsymbol{S}_t^\star \mid \boldsymbol{z}_{1:t})$, because the current pair $(\boldsymbol{k}_t, \boldsymbol{v}_t)$ has not been used. The covariance $\widehat{\boldsymbol{P}}_t$ is not a point estimate; it is the predicted uncertainty around that mean. The predicted read and innovation are

$$\widehat{\boldsymbol{v}}_t = \widehat{\boldsymbol{S}}_t^\intercal \boldsymbol{k}_t, \qquad \boldsymbol{\delta}_t = \boldsymbol{v}_t - \widehat{\boldsymbol{v}}_t. \tag{7}$$

This residual form is the matrix version of the standard Kalman correction. To see it, first look at one value coordinate j. Let $\boldsymbol{s}_{t,j}$ be column j of the memory and let $v_{t,j}$ be coordinate j of the observed value. The measurement model is

$$v_{t,j} = \boldsymbol{k}_t^\intercal \boldsymbol{s}_{t,j}^\star + e_{t,j}, \qquad e_{t,j} \sim \mathcal{N}(0, r_t). \tag{8}$$

Before this measurement is used, the predicted distribution for this column is

$$\boldsymbol{s}^{\star}_{t,j} \mid \boldsymbol{z}_{1:t-1} \sim \mathcal{N}(\widehat{\boldsymbol{s}}_{t,j}, \widehat{\boldsymbol{P}}_t). \tag{9}$$

The observation $v_{t,j}$ is one scalar. Its predicted mean is $\boldsymbol{k}_t^{\intercal}\widehat{\boldsymbol{s}}_{t,j}$, so the part of the observation not already predicted by the prior is the scalar residual

$$\delta_{t,j} = v_{t,j} - \boldsymbol{k}_t^{\intercal}\widehat{\boldsymbol{s}}_{t,j}. \tag{10}$$

For a joint Gaussian pair (x, y) with scalar y, the conditional mean is

$$\mathbb{E}[x \mid y] = \mathbb{E}[x] + \operatorname{Cov}(x, y)\operatorname{Var}(y)^{-1}\big(y - \mathbb{E}[y]\big).$$

Apply this identity with $x = \boldsymbol{s}^{\star}_{t,j}$ and $y = v_{t,j}$. Under (9)–(8),

$$\operatorname{Cov}(\boldsymbol{s}^{\star}_{t,j}, v_{t,j}) = \widehat{\boldsymbol{P}}_t\boldsymbol{k}_t, \qquad \operatorname{Var}(v_{t,j}) = \boldsymbol{k}_t^{\intercal}\widehat{\boldsymbol{P}}_t\boldsymbol{k}_t + r_t. \tag{11}$$

Therefore the posterior mean is

$$\boldsymbol{s}_{t,j} = \widehat{\boldsymbol{s}}_{t,j} + \boldsymbol{\kappa}_t\,\delta_{t,j}. \tag{12}$$

where

$$\boldsymbol{\kappa}_t = \frac{\widehat{\boldsymbol{P}}_t\boldsymbol{k}_t}{\boldsymbol{k}_t^{\intercal}\widehat{\boldsymbol{P}}_t\boldsymbol{k}_t + r_t}. \tag{13}$$

The same $\boldsymbol{\kappa}_t$ applies to every value coordinate because the model uses the same key-space covariance $\widehat{\boldsymbol{P}}_t$ and the same observation noise r_t for every coordinate of $\boldsymbol{v}_t$. Stacking (12) over $j = 1, \ldots, d_v$ gives the rank-one residual write

$$\boldsymbol{S}_t = \widehat{\boldsymbol{S}}_t + \boldsymbol{\kappa}_t\big(\boldsymbol{v}_t - \widehat{\boldsymbol{S}}_t^{\intercal}\boldsymbol{k}_t\big)^{\intercal}. \tag{14}$$

Thus the residual write does not come from DeltaNet first; it comes from the Kalman posterior-mean update $x^{+} = x^{-} + K(y - Hx^{-})$. DeltaNet later appears as the fixed-gain special case where $\boldsymbol{\kappa}_t$ is forced to be proportional to $\boldsymbol{k}_t$. For a candidate write direction $\boldsymbol{\kappa}_t$, the posterior covariance induced by this write is

$$\boldsymbol{P}_t(\boldsymbol{\kappa}_t) = (\boldsymbol{I} - \boldsymbol{\kappa}_t\boldsymbol{k}_t^{\intercal})\widehat{\boldsymbol{P}}_t(\boldsymbol{I} - \boldsymbol{\kappa}_t\boldsymbol{k}_t^{\intercal})^{\intercal} + r_t\boldsymbol{\kappa}_t\boldsymbol{\kappa}_t^{\intercal}. \tag{15}$$

The Kalman write direction is the minimizer of total posterior uncertainty,

$$\boldsymbol{\kappa}_t^{\star} = \arg\min_{\boldsymbol{\kappa}_t} \operatorname{Tr}(\boldsymbol{P}_t(\boldsymbol{\kappa}_t))\,. \tag{16}$$

Expanding the trace gives

$$\operatorname{Tr}(\boldsymbol{P}_t(\boldsymbol{\kappa}_t)) = \operatorname{Tr}(\widehat{\boldsymbol{P}}_t) - 2\boldsymbol{\kappa}_t^{\intercal}\widehat{\boldsymbol{P}}_t\boldsymbol{k}_t + \big(r_t + \boldsymbol{k}_t^{\intercal}\widehat{\boldsymbol{P}}_t\boldsymbol{k}_t\big)\boldsymbol{\kappa}_t^{\intercal}\boldsymbol{\kappa}_t. \tag{17}$$

Taking the gradient with respect to $\boldsymbol{\kappa}_t$ and setting it to zero yields

$$\boxed{\boldsymbol{\kappa}_t = \frac{\widehat{\boldsymbol{P}}_t\boldsymbol{k}_t}{r_t + \boldsymbol{k}_t^{\intercal}\widehat{\boldsymbol{P}}_t\boldsymbol{k}_t}}. \tag{18}$$

Therefore the exact Kalman Associative Memory update is

$$\begin{aligned}
&\widehat{\boldsymbol{S}}_t = \boldsymbol{D}_t\boldsymbol{S}_{t-1}, && \widehat{\boldsymbol{P}}_t = \boldsymbol{D}_t\boldsymbol{P}_{t-1}\boldsymbol{D}_t^{\intercal} + \boldsymbol{\Omega}_t,\\
&\boldsymbol{\kappa}_t = \frac{\widehat{\boldsymbol{P}}_t\boldsymbol{k}_t}{r_t + \boldsymbol{k}_t^{\intercal}\widehat{\boldsymbol{P}}_t\boldsymbol{k}_t}, && \boldsymbol{S}_t = \widehat{\boldsymbol{S}}_t + \boldsymbol{\kappa}_t\big(\boldsymbol{v}_t - \widehat{\boldsymbol{S}}_t^{\intercal}\boldsymbol{k}_t\big)^{\intercal},\\
&\boldsymbol{P}_t = (\boldsymbol{I} - \boldsymbol{\kappa}_t\boldsymbol{k}_t^{\intercal})\widehat{\boldsymbol{P}}_t.
\end{aligned} \tag{19}$$

This is the reference update. The remaining question is which part of it can be made tractable and scan-computable without losing the link between uncertainty, transition, and write strength.

2 Dense covariance and dense information before approximation

Before introducing any diagonal restriction, keep the covariance as a full dense matrix $\boldsymbol{P}_t \in \mathbb{R}^{d_k \times d_k}$. The prediction step is exactly

$$\widehat{\boldsymbol{P}}_t = \boldsymbol{D}_t \boldsymbol{P}_{t-1} \boldsymbol{D}_t^\intercal + \boldsymbol{\Omega}_t. \tag{20}$$

After the measurement at key $\boldsymbol{k}_t$, the Kalman gain and covariance update are

$$\boldsymbol{\kappa}_t = \frac{\widehat{\boldsymbol{P}}_t \boldsymbol{k}_t}{r_t + \boldsymbol{k}_t^\intercal \widehat{\boldsymbol{P}}_t \boldsymbol{k}_t}, \qquad \boldsymbol{P}_t = (\boldsymbol{I} - \boldsymbol{\kappa}_t \boldsymbol{k}_t^\intercal)\widehat{\boldsymbol{P}}_t. \tag{21}$$

Substituting the gain into the covariance update gives the dense posterior covariance explicitly:

$$\begin{aligned} \boldsymbol{P}_t &= \widehat{\boldsymbol{P}}_t - \frac{\widehat{\boldsymbol{P}}_t \boldsymbol{k}_t}{r_t + \boldsymbol{k}_t^\intercal \widehat{\boldsymbol{P}}_t \boldsymbol{k}_t} \boldsymbol{k}_t^\intercal \widehat{\boldsymbol{P}}_t \\ &= \widehat{\boldsymbol{P}}_t - \frac{\widehat{\boldsymbol{P}}_t \boldsymbol{k}_t \boldsymbol{k}_t^\intercal \widehat{\boldsymbol{P}}_t}{r_t + \boldsymbol{k}_t^\intercal \widehat{\boldsymbol{P}}_t \boldsymbol{k}_t}. \end{aligned} \tag{22}$$

This is still exact and dense.

Now define dense information matrices

$$\boldsymbol{C}_t = \boldsymbol{P}_t^{-1}, \qquad \widehat{\boldsymbol{C}}_t = \widehat{\boldsymbol{P}}_t^{-1}.$$

The rank-one Woodbury identity gives

$$\left(\widehat{\boldsymbol{C}}_t + \frac{1}{r_t} \boldsymbol{k}_t \boldsymbol{k}_t^\intercal\right)^{-1} = \widehat{\boldsymbol{P}}_t - \frac{\widehat{\boldsymbol{P}}_t \boldsymbol{k}_t \boldsymbol{k}_t^\intercal \widehat{\boldsymbol{P}}_t}{r_t + \boldsymbol{k}_t^\intercal \widehat{\boldsymbol{P}}_t \boldsymbol{k}_t}. \tag{23}$$

The right-hand side is exactly (22). Therefore the same dense measurement update in information form is

$$\boxed{\boldsymbol{C}_t = \widehat{\boldsymbol{C}}_t + \frac{1}{r_t} \boldsymbol{k}_t \boldsymbol{k}_t^\intercal.} \tag{24}$$

So if we allow dense $\boldsymbol{P}_t$ or dense $\boldsymbol{C}_t$, there is no approximation here. The measurement adds a rank-one dense information matrix.

What becomes difficult. The dense route is mathematically clean but not the desired linear-attention primitive. First, the state itself costs $O(d_k^2)$ per head instead of $O(d_k)$. Computing the gain requires the dense matrix-vector product $\widehat{\boldsymbol{P}}_t \boldsymbol{k}_t$ and the quadratic form $\boldsymbol{k}_t^\intercal \widehat{\boldsymbol{P}}_t \boldsymbol{k}_t$, which are $O(d_k^2)$ per token. Second, the covariance recursion is sequential: the gain at time t depends on the dense posterior uncertainty produced by all previous keys. Third, dense information form only makes the measurement update simple. The prediction step becomes

$$\widehat{\boldsymbol{C}}_t = \left(\boldsymbol{D}_t \boldsymbol{C}_{t-1}^{-1} \boldsymbol{D}_t^\intercal + \boldsymbol{\Omega}_t\right)^{-1}, \tag{25}$$

which requires an inverse when the process noise $\boldsymbol{\Omega}_t$ is additive. Thus dense $\boldsymbol{C}_t$ is easy to update after a measurement but hard to predict through process noise, while dense $\boldsymbol{P}_t$ is easy to predict but expensive to update and use for gains.

What is $\widehat{C}_t$ used for? The Kalman gain itself is naturally written in terms of the predicted covariance:

$$\boldsymbol{\kappa}_t = \frac{\widehat{\boldsymbol{P}}_t \boldsymbol{k}_t}{r_t + \boldsymbol{k}_t^\intercal \widehat{\boldsymbol{P}}_t \boldsymbol{k}_t}. \tag{26}$$

Thus the gain does not need $\widehat{\boldsymbol{C}}_t$ as a matrix; it needs the vector

$$\boldsymbol{x}_t = \widehat{\boldsymbol{P}}_t \boldsymbol{k}_t = \widehat{\boldsymbol{C}}_t^{-1} \boldsymbol{k}_t, \qquad \boldsymbol{k}_t^\intercal \widehat{\boldsymbol{P}}_t \boldsymbol{k}_t = \boldsymbol{k}_t^\intercal \boldsymbol{x}_t. \tag{27}$$

If $\widehat{\boldsymbol{C}}_t$ is dense, computing $\boldsymbol{x}_t$ means solving the linear system $\widehat{\boldsymbol{C}}_t \boldsymbol{x}_t = \boldsymbol{k}_t$, which is not cheaper than dense covariance algebra in general.

One can compute the gain ingredients directly from $\boldsymbol{C}_{t-1}$ without explicitly forming $\widehat{\boldsymbol{C}}_t$:

$$\begin{aligned} \widehat{\boldsymbol{P}}_t \boldsymbol{k}_t &= \big(\boldsymbol{D}_t \boldsymbol{C}_{t-1}^{-1} \boldsymbol{D}_t^\intercal + \boldsymbol{\Omega}_t\big) \boldsymbol{k}_t \\ &= \boldsymbol{D}_t \boldsymbol{C}_{t-1}^{-1} (\boldsymbol{D}_t^\intercal \boldsymbol{k}_t) + \boldsymbol{\Omega}_t \boldsymbol{k}_t. \end{aligned} \tag{28}$$

This is exact if the solve with $\boldsymbol{C}_{t-1}$ is exact. It is useful conceptually because it shows that $\widehat{\boldsymbol{C}}_t$ is mainly a predicted information state for the measurement update and scan recursion, not a required intermediate for the gain. In the diagonal case, (28) becomes coordinatewise:

$$\widehat{p}_{t,i} k_{t,i} = \left(\frac{\alpha_{t,i}^2}{c_{t-1,i}} + \omega_{t,i} \right) k_{t,i}, \qquad \boldsymbol{k}_t^\intercal \widehat{\boldsymbol{P}}_t \boldsymbol{k}_t = \sum_i \left(\frac{\alpha_{t,i}^2}{c_{t-1,i}} + \omega_{t,i} \right) k_{t,i}^2. \tag{29}$$

Equivalently, $\widehat{c}_{t,i} = 1/\widehat{p}_{t,i}$ is used when updating the information state after the measurement.

Can we parameterize $\boldsymbol{\Omega}_t \boldsymbol{k}_t$ directly? The term $\boldsymbol{\Omega}_t \boldsymbol{k}_t$ is the process-noise contribution to the gain numerator. Define

$$\boldsymbol{\xi}_t \triangleq \boldsymbol{\Omega}_t \boldsymbol{k}_t.$$

Then the gain ingredients can be written as

$$\begin{aligned} \widehat{\boldsymbol{P}}_t \boldsymbol{k}_t &= \boldsymbol{D}_t \boldsymbol{C}_{t-1}^{-1} (\boldsymbol{D}_t^\intercal \boldsymbol{k}_t) + \boldsymbol{\xi}_t, \\ \boldsymbol{k}_t^\intercal \widehat{\boldsymbol{P}}_t \boldsymbol{k}_t &= (\boldsymbol{D}_t^\intercal \boldsymbol{k}_t)^\intercal \boldsymbol{C}_{t-1}^{-1} (\boldsymbol{D}_t^\intercal \boldsymbol{k}_t) + \boldsymbol{k}_t^\intercal \boldsymbol{\xi}_t. \end{aligned} \tag{30}$$

So for the gain alone, it is enough to know the vector action $\boldsymbol{\xi}_t$ and the scalar $\boldsymbol{k}_t^\intercal \boldsymbol{\xi}_t$; one does not need the full matrix $\boldsymbol{\Omega}_t$.

However, $\boldsymbol{\xi}_t$ is not automatically a valid process covariance. To remain Kalman-consistent, there must exist a symmetric positive semidefinite $\boldsymbol{\Omega}_t$ such that $\boldsymbol{\xi}_t = \boldsymbol{\Omega}_t \boldsymbol{k}_t$ and $\boldsymbol{k}_t^\intercal \boldsymbol{\xi}_t = \boldsymbol{k}_t^\intercal \boldsymbol{\Omega}_t \boldsymbol{k}_t \geq 0$. In the diagonal case this means

$$\boldsymbol{\xi}_t = \boldsymbol{\omega}_t \odot \boldsymbol{k}_t, \qquad \boldsymbol{\omega}_t \geq 0, \qquad \boldsymbol{k}_t^\intercal \boldsymbol{\xi}_t = \sum_i \omega_{t,i} k_{t,i}^2.$$

This is equivalent to parameterizing the diagonal process noise $\boldsymbol{\omega}_t$; it does not change the uncertainty scan. If instead we parameterize an arbitrary vector $\boldsymbol{\xi}_t$ and a positive scalar denominator contribution directly, the gain can be made easier to compute and remains input-only once the prefix information state is known. But then the update is no longer the exact Kalman covariance recursion unless we also define a compatible $\boldsymbol{\Omega}_t$ and update the uncertainty state with it. In that case it should be viewed as a learned gain approximation rather than a full process-noise model.

Options from the dense information prediction. Strictly, (25) defines the predicted information $\widehat{\boldsymbol{C}}_t$, not the posterior information $\boldsymbol{C}_t$. From this equation, the available paths are:

1. **Stay exact and dense.** Compute $\widehat{\boldsymbol{C}}_t = (\boldsymbol{D}_t \boldsymbol{C}_{t-1}^{-1} \boldsymbol{D}_t^\intercal + \boldsymbol{\Omega}_t)^{-1}$ directly. This keeps the Kalman filter exact, but requires dense inverses or dense linear solves.

2. **Use covariance for prediction.** Keep the uncertainty state in covariance form rather than information form. Then the process step is immediate:

$$\widehat{\boldsymbol{P}}_t = \boldsymbol{D}_t \boldsymbol{P}_{t-1} \boldsymbol{D}_t^\intercal + \boldsymbol{\Omega}_t.$$

   This avoids the difficult inverse in (25). The gain is then computed from this predicted covariance:

$$\boldsymbol{\kappa}_t = \frac{\widehat{\boldsymbol{P}}_t \boldsymbol{k}_t}{r_t + \boldsymbol{k}_t^\intercal \widehat{\boldsymbol{P}}_t \boldsymbol{k}_t}.$$

   After the measurement, the covariance state for the next token is

$$\boldsymbol{P}_t = (\boldsymbol{I} - \boldsymbol{\kappa}_t \boldsymbol{k}_t^\intercal) \widehat{\boldsymbol{P}}_t.$$

   Therefore the recurrence is

$$\boldsymbol{P}_{t-1} \longrightarrow \widehat{\boldsymbol{P}}_t \longrightarrow \boldsymbol{\kappa}_t \longrightarrow \boldsymbol{P}_t.$$

   If implemented literally, this recurrence is sequential. If we ignore storage and compute cost, however, the dense covariance recursion is parallelizable in principle. Each token defines a Riccati map $\boldsymbol{P}_t = F_t(\boldsymbol{P}_{t-1})$, and prefix covariances are

$$\boldsymbol{P}_t = (F_t \circ F_{t-1} \circ \cdots \circ F_1)(\boldsymbol{P}_0).$$

   Function composition is associative. More concretely, when $\boldsymbol{D}_t$ is invertible, the prediction and measurement updates are matrix fractional maps. The prediction can be written as

$$\widehat{\boldsymbol{P}}_t = (\boldsymbol{D}_t \boldsymbol{P}_{t-1} + \boldsymbol{\Omega}_t \boldsymbol{D}_t^{-\intercal})(\boldsymbol{D}_t^{-\intercal})^{-1},$$

   and the measurement update can be written as

$$\boldsymbol{P}_t = \widehat{\boldsymbol{P}}_t \left(\boldsymbol{I} + \frac{1}{r_t} \boldsymbol{k}_t \boldsymbol{k}_t^\intercal \widehat{\boldsymbol{P}}_t \right)^{-1}.$$

   These fractional maps compose by multiplying their block-matrix representations, so the dense uncertainty prefix can be computed by a scan in principle. The issue is therefore not mathematical associativity. The issue is that this is a dense Riccati scan, not the cheap affine memory scan used by linear attention; after the scan one still has to recover $\widehat{\boldsymbol{P}}_t$ and form $\widehat{\boldsymbol{P}}_t \boldsymbol{k}_t$ for every token to get the gains. Converting to information would mean forming $\widehat{\boldsymbol{C}}_t = \widehat{\boldsymbol{P}}_t^{-1}$, which is another dense inverse, so this path usually stays in covariance form.

   The memory cost is the main practical obstacle. At autoregressive inference time, a dense covariance filter stores one $\boldsymbol{P}_t \in \mathbb{R}^{d_k \times d_k}$ per head, i.e. $O(d_k^2)$ uncertainty state instead of the $O(d_k)$ state of a diagonal tracker. In parallel training, a dense Riccati scan must materialize either dense prefix covariances or dense fractional-map scan elements for many tokens. Each token's fractional map is represented by a $2d_k \times 2d_k$ block matrix, equivalently four dense $d_k \times d_k$ blocks, so the scan state is still $O(d_k^2)$ per token. Across a length-T sequence this is $O(Td_k^2)$ per head just for the uncertainty scan, before storing the usual key, value, gain, and memory-scan quantities. The diagonal approximation reduces this uncertainty-scan storage to $O(Td_k)$.

3. **Remove additive process noise.** If $\boldsymbol{\Omega}_t = 0$ and $\boldsymbol{D}_t$ is invertible, then the information prediction is exact and simple:
$$\widehat{\boldsymbol{C}}_t = \boldsymbol{D}_t^{-\intercal} \boldsymbol{C}_{t-1} \boldsymbol{D}_t^{-1}.$$
This is tractable algebraically, but it removes the additive uncertainty growth that represents drift.

4. **Restrict to a diagonal family.** If $\boldsymbol{D}_t$, $\boldsymbol{\Omega}_t$, and $\boldsymbol{C}_{t-1}$ are diagonal, the inverse is coordinate-wise:
$$\widehat{c}_{t,i} = \frac{c_{t-1,i}}{\alpha_{t,i}^2 + \omega_{t,i} c_{t-1,i}}.$$
This is the tractable path used below, but the following measurement update must be projected back to diagonal.

5. **Use structured covariance.** Keep blocks, diagonal-plus-low-rank factors, or chunkwise dense states. These approximate the dense equation more closely than a diagonal vector, but their scan and gain computations are more expensive.

This is the point where tractability enters. The diagonal state below is not needed for the Kalman derivation; it is a computational restriction that keeps the uncertainty state linear in d_k and makes a scan-compatible approximation possible.

3 Diagonal information as the tractable state

Because the dense recursions above are too expensive for a linear-attention layer, we restrict the uncertainty state to the space of diagonal covariance matrices, the simplest family that is still more expressive than a scalar gain:

$$\boldsymbol{P}_t \in \{\mathrm{diag}(\boldsymbol{p}) : \boldsymbol{p} \in \mathbb{R}_+^{d_k}\}, \qquad \boldsymbol{P}_t = \mathrm{diag}(\boldsymbol{p}_t), \qquad \boldsymbol{C}_t = \boldsymbol{P}_t^{-1} = \mathrm{diag}(\boldsymbol{c}_t). \tag{31}$$

Here $\boldsymbol{p}_t$ is per-channel uncertainty and $\boldsymbol{c}_t$ is per-channel information. The information parameterization $\boldsymbol{C}_t$ is useful, but it is only one possible way to move forward; the exact object under the diagonal restriction is still the covariance prediction below.

3.1 Transition-aware diagonal prediction

With a diagonal process model $\boldsymbol{D}_t = \mathrm{diag}(\boldsymbol{\alpha}_t)$ and diagonal process noise $\boldsymbol{\Omega}_t = \mathrm{diag}(\boldsymbol{\omega}_t)$, the Kalman covariance prediction remains pointwise:

$$\widehat{\boldsymbol{p}}_t = \boldsymbol{\alpha}_t^2 \odot \boldsymbol{p}_{t-1} + \boldsymbol{\omega}_t. \tag{21}$$

The gain is then the diagonal approximation of the Kalman gain,

$$\boldsymbol{\kappa}_t = \frac{\widehat{\boldsymbol{p}}_t \odot \boldsymbol{k}_t}{r_t + \sum_i \widehat{p}_{t,i} k_{t,i}^2}. \tag{22}$$

Unlike the fixed-gain delta rule, in the exact prediction (21) the gain $\boldsymbol{\kappa}_t$ depends on the same process model that predicts the memory: if a channel is forgotten or assigned high process noise, its uncertainty changes before the write is formed. How much of this coupling survives once the recursion is made scan-computable depends on the parameterization.

Per channel, the covariance predict is

$$\boxed{\widehat{p}_t = \alpha_t^2 p_{t-1} + \omega_t.} \tag{67}$$

This is the diagonal form of (21). It is the only exact predicted variance; every expression below is (67) written in different variables or followed by an approximation.

Options after the exact predicted variance

At this point the derivation is still exact under the diagonal covariance restriction. There are several ways to go forward:

1. **Keep covariance variables.** Track $\boldsymbol{p}_t$ directly and compute $\widehat{\boldsymbol{p}}_t$ by (21). This is closest to the Kalman gain, but the posterior covariance update after measurement is not diagonal unless we project.
2. **Use information variables.** Track $\boldsymbol{c}_t = \boldsymbol{p}_t^{-1}$. This makes the Gaussian measurement update additive in information form and can lead to a scan-computable recursion. This is one option, not the only one.
3. **Project to an isotropic scalar.** Replace $\boldsymbol{p}_t$ by $b_t\mathbf{1}$ or $\boldsymbol{P}_t$ by $b_t\boldsymbol{I}$. This preserves the delta-rule write direction $\boldsymbol{\kappa}_t \propto \boldsymbol{k}_t$, but loses per-channel uncertainty.
4. **Use a richer structured family.** Track block-diagonal, diagonal-plus-low-rank, or chunk-boundary covariance states. These keep more correlations than a diagonal vector, but they are more expensive and require a different scan design.
5. **Freeze or approximate the gain.** Compute gains from input-only or chunkwise prefix statistics and then use the standard affine memory scan. This is an implementation choice, not an exact Kalman covariance recursion.

4 Measurement boundary after choosing a diagonal state

From the dense derivation above, the measurement update in information variables is (24). Now impose the diagonal state restriction. If $\widehat{\boldsymbol{C}}_t = \mathrm{diag}(\widehat{\boldsymbol{c}}_t)$, then the measurement still gives

$$\boldsymbol{C}_t = \mathrm{diag}(\widehat{\boldsymbol{c}}_t) + \frac{1}{r_t}\boldsymbol{k}_t\boldsymbol{k}_t^\intercal. \tag{32}$$

The rank-one term is dense for a general key $\boldsymbol{k}_t$, so the posterior information matrix is generally not diagonal:

$$\boldsymbol{C}_t \notin \{\mathrm{diag}(\boldsymbol{c}) : \boldsymbol{c} \in \mathbb{R}_+^{d_k}\}.$$

A vector state $\boldsymbol{c}_t$ cannot represent this posterior exactly. This is where exactness stops for the diagonal family: any diagonal update after (32) is a projection or approximation.

Options at the measurement boundary

After reaching the dense posterior in (32), the available choices are:

1. **Keep the exact dense posterior.** This remains Kalman-exact, but it loses the diagonal state and is not the intended tractable linear-attention update.

2. **Literal diagonal projection.** Keep only the diagonal of $\boldsymbol{k}_t\boldsymbol{k}_t^\intercal/r_t$, giving $\boldsymbol{c}_t = \widehat{\boldsymbol{c}}_t + (\boldsymbol{k}_t \odot \boldsymbol{k}_t)/r_t$. This is the literal diagonal of the exact Fisher term, but it is poorly scaled for normalized high-dimensional keys.

3. **Calibrated diagonal projection.** Standardize each key coordinate before taking the diagonal Fisher increment, giving the dimension-stable update derived below.

4. **Isotropic projection.** Average the calibrated diagonal information into one scalar. This keeps a scalar uncertainty and recovers a gain direction proportional to $\boldsymbol{k}_t$.

5. **Structured projection.** Keep block-diagonal or low-rank pieces of the rank-one Fisher term. This is closer to the exact posterior but more expensive than a diagonal vector.

5 Calibrated diagonal information update

If we choose a diagonal vector state, we need to replace the dense Fisher matrix $\boldsymbol{k}_t\boldsymbol{k}_t^\intercal/r_t$ by a diagonal increment. The literal diagonal increment is

$$\frac{1}{r_t}\boldsymbol{k}_t \odot \boldsymbol{k}_t.$$

For ℓ_2-normalized keys with spread-out energy, a typical coordinate satisfies $\mathbb{E}[k_{t,i}^2] \approx 1/d_k$. Therefore the literal per-channel increment has size

$$\mathbb{E}\left[\frac{k_{t,i}^2}{r_t}\right] \approx \frac{1}{d_k r_t},$$

which vanishes as the key dimension grows. That scaling makes the diagonal tracker depend on the arbitrary representation dimension rather than on the observation reliability r_t.

To get a dimension-stable per-channel Fisher increment, use the standardized coordinate

$$\widetilde{k}_{t,i} = \sqrt{d_k}\, k_{t,i}.$$

For normalized spread-out keys,

$$\mathbb{E}[\widetilde{k}_{t,i}^2] = d_k \mathbb{E}[k_{t,i}^2] \approx 1.$$

The scalar Fisher information contributed by coordinate i is therefore

$$\frac{\widetilde{k}_{t,i}^2}{r_t} = \frac{d_k}{r_t} k_{t,i}^2.$$

This gives the calibrated diagonal update

$$\boxed{\boldsymbol{c}_t = \widehat{\boldsymbol{c}}_t + \frac{d_k}{r_t}\,\boldsymbol{k}_t \odot \boldsymbol{k}_t.} \tag{33}$$

Equivalently, per channel,

$$c_{t,i} = \widehat{c}_{t,i} + \frac{d_k}{r_t} k_{t,i}^2. \tag{34}$$

Where approximation enters

The prediction

$$\widehat{p}_{t,i} = \alpha_{t,i}^2 p_{t-1,i} + \omega_{t,i}$$

is exact under the diagonal covariance restriction. The dense information update

$$\boldsymbol{C}_t = \widehat{\boldsymbol{C}}_t + \boldsymbol{k}_t \boldsymbol{k}_t^\intercal / r_t$$

is exact for the linear–Gaussian measurement. The calibrated diagonal update (33) is not the exact posterior; it is a dimension-stable diagonal projection of the measurement information. From this point onward, any scan-computable diagonal recursion, including an information-form or Möbius recurrence, is an approximation to the exact dense Kalman posterior.

Options after the calibrated projection

Once the calibrated diagonal update is chosen, the remaining design options are:

1. **Free additive process noise.** Keep $\omega_{t,i}$ as an input-only additive variance. In information variables this produces a per-channel linear-fractional recurrence that can be scanned with 2×2 matrices.
2. **State-proportional process noise.** Choose $\omega_t \propto p_{t-1}$ so the information recursion becomes affine. This is cheaper, but it restricts the process-noise model.
3. **Isotropic information.** Average the diagonal information into one scalar if the goal is to preserve the delta-rule write direction exactly.
4. **Richer covariance families.** Move beyond the diagonal projection with blocks, low-rank terms, or chunkwise dense updates if the diagonal approximation is too weak.

Note: language-modeling objective

The trace objective in (16) is a local Bayesian criterion for the associative-memory update. Under the linear–Gaussian model, it chooses the Kalman/MMSE write direction by minimizing posterior covariance. This is a useful adaptive-gain prior, but it is not the objective optimized by an autoregressive language model. A language model is trained to minimize future next-token negative log likelihood under a finite memory budget.

For the language-modeling use case, the more aligned local question is therefore:

> *Choose the write gain that most improves future next-token prediction under limited memory capacity.*

Writing

$$\boldsymbol{S}_t(\boldsymbol{\kappa}) = \widehat{\boldsymbol{S}}_t + \boldsymbol{\kappa}\big(\boldsymbol{v}_t - \widehat{\boldsymbol{S}}_t^\intercal \boldsymbol{k}_t\big)^\intercal,$$

a direct idealization is

$$\boldsymbol{\kappa}_t^\star \approx \arg\min_{\boldsymbol{\kappa}} \mathbb{E}_{1 \le h \le H} \left[\ell_{\mathrm{LM}}(x_{t+h} \mid x_{<t+h}, \boldsymbol{S}_t(\boldsymbol{\kappa}))\right] + \lambda_{\mathrm{int}}\, \mathcal{I}_t(\boldsymbol{\kappa}), \tag{35}$$

where $\mathcal{I}_t$ measures interference with useful contents already stored in the memory. This objective is not meant to be solved literally inside the layer. It clarifies what the Kalman update should approximate if it is to be useful for language modeling.

Short-horizon predictive loss. The most direct proxy is to prefer writes that reduce cross entropy over a short future horizon:

$$\boldsymbol{\kappa}_t \quad \text{should reduce} \quad \ell_{\mathrm{LM}}(x_{t+1:t+H} \mid x_{\le t}, \boldsymbol{S}_t).$$

This says that a good write is one that helps future prediction, not merely one that reduces geometric uncertainty.

Future-read reconstruction. The memory should store information that future queries will actually read. A corresponding proxy is

$$\mathbb{E}_{\tau > t}\left[\|\boldsymbol{S}_t(\boldsymbol{\kappa})^{\intercal}\boldsymbol{q}_\tau - \boldsymbol{y}_\tau\|_2^2\right],$$

where $\boldsymbol{q}_\tau$ denotes a future read query and $\boldsymbol{y}_\tau$ denotes the target content for that query. This differs from $\mathrm{Tr}(\boldsymbol{P}_t)$ because language modeling does not value all key-space directions equally; it values directions that future tokens will query.

Surprise-weighted writing. A language-model memory should write more when the current token contributes useful new information. Thus the gain should depend not only on key-space uncertainty, but also on a causal surprise signal:

$$\text{write strength} \propto \text{useful surprise} \times \text{retrievability} \times \text{non-redundancy}.$$

The associative-memory innovation is one local, non-leaking proxy:

$$\boldsymbol{\delta}_t = \boldsymbol{v}_t - \widehat{\boldsymbol{S}}_t^{\intercal}\boldsymbol{k}_t, \qquad s_t = \mathrm{sg}\left(\frac{1}{d_v}\|\boldsymbol{\delta}_t\|_2^2\right),$$

where $\mathrm{sg}(\cdot)$ denotes stop-gradient. A conservative implementation can let s_t modulate the Kalman confidence parameters:

$$r_t = r_{\min} + \mathrm{softplus}(\rho_t - a\,\widetilde{s}_t), \qquad \omega_{t,i} = \omega_{\min} + \mathrm{softplus}(\eta_{t,i} + b_i\,\widetilde{s}_t).$$

Large innovation can then decrease r_t or increase ω_t, allowing a larger update. The modulation coefficients should be initialized near zero and used with floors or clamps, so the model starts as ordinary KLA and learns whether surprise-dependent plasticity helps.

Capacity-aware utility. Because the recurrent state is small relative to the sequence, every write competes with existing contents. A language-model-aligned update should therefore optimize a net utility:

$$\text{future CE improvement} - \text{interference cost}.$$

This is the most relevant framing for KLA and DeltaNet-style memories. A write is useful only if it improves future recall or prediction more than it harms information already stored for future reads.

In this framing, Kalman uncertainty minimization provides a principled adaptive-gain prior. For language models, the actual selection pressure is predictive memory utility: writes should reduce future language-model loss under capacity and interference constraints. Signals derived from top-level entropy or realized negative log likelihood must be used causally, for example to modulate the next update, so that the memory update does not leak the target token into the current hidden state.